الأكاديمية الوطنية لتقنية المعلومات
NATIONAL INFORMATION TECHNOLOGY ACADEMY

NITA

LAB MANUAL: CLOUD COMPUTING FOR DATA SCIENCE

8–12 Nov 2026

Roadmap

Part 1 · Get the app and read the code

Part 2 · Store the code on GitHub

Part 3 · Deploy on a PaaS

Part 4 · Update and redeploy

Part 5 · Who managed what?

Part 6 · Reflection 💬

At the end of this lab session you will be able to:

- *Put a ready-made data app into a GitHub repository (using a SaaS).*
- *Deploy it to a platform-as-a-service and share a public link.*
- *Change the code and watch the platform redeploy it automatically.*
- *Explain which layers you managed and which the platform managed.*



Lab 2: Deploy a Data App on a PaaS

Lab 2: Deploy a Data App on a PaaS

Estimated time	60–75 minutes
Files	app.py, requirements.txt, Admission_Predict_Ver1.1.csv (Blackboard)
Tools	Web browser, GitHub (free account), Streamlit Community Cloud

Browser only *— nothing to install. Create a free GitHub account before the lab.*

Before you start

- Download the three files from Blackboard into one folder.
- Use the same GitHub account for GitHub and Streamlit Community Cloud.
- Exercise slides are marked in amber: try them yourself first, then compare with the solution shown.



Lab 2: Part 1 · Get the app and read the code

Lab 2: Part 1 · The three files

app.py is the app. The CSV is the data. requirements.txt lists the Python packages the platform must install.

File

```
# requirements.txt
streamlit
pandas
matplotlib
```

Lab 2: Part 1 · Read the key parts of app.py

@st.cache_data loads the CSV once. The slider filters the data. The metrics and chart show the result.

File

```
@st.cache_data
def load_data():
    df = pd.read_csv("Admission_Predict_Ver1.1.csv")
    df.columns = df.columns.str.strip()
    return df

min_cgpa = st.sidebar.slider("Minimum CGPA", 6.5, 10.0, 8.0, 0.1)
view = df[df["CGPA"] >= min_cgpa]

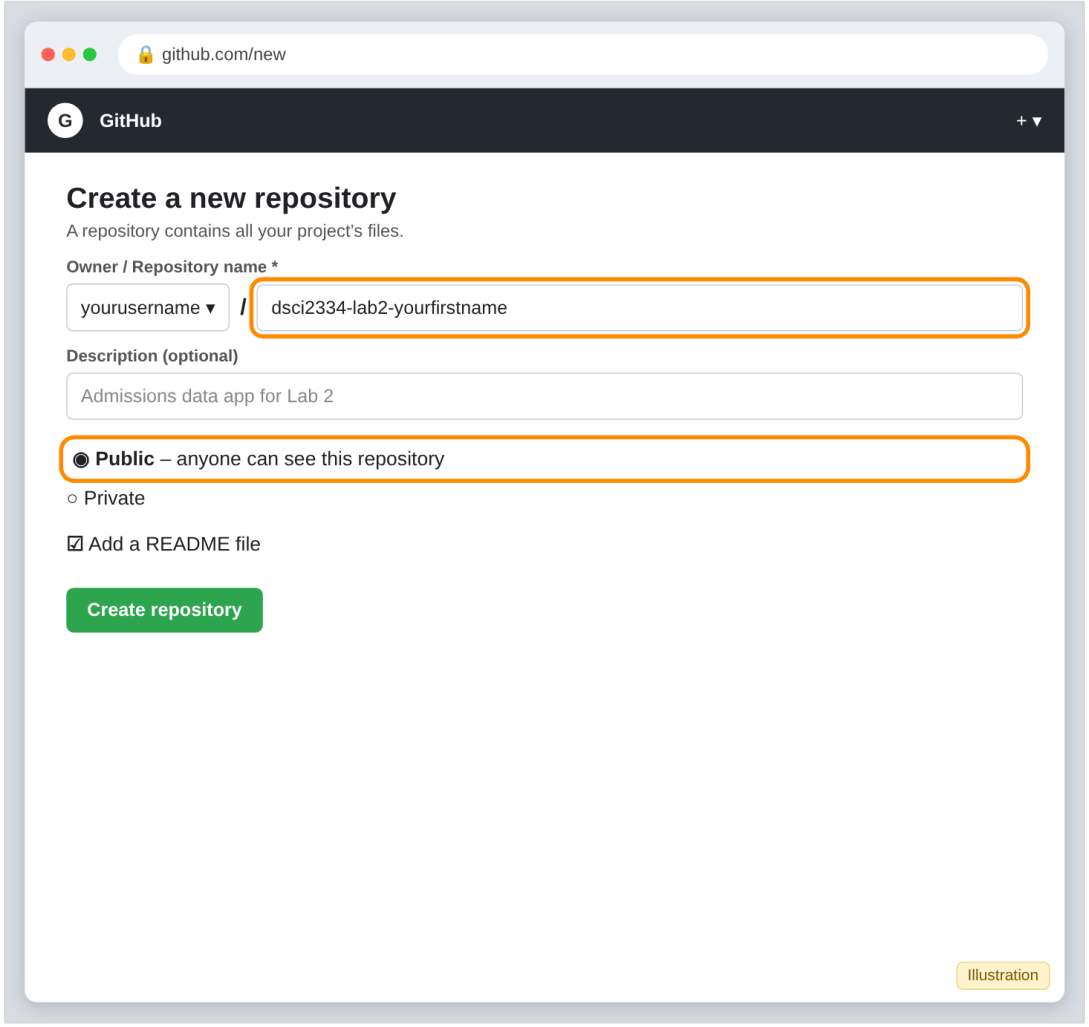
c1, c2, c3 = st.columns(3)
c1.metric("Students", len(view))
st.pyplot(fig)
```



Lab 2: Part 2 · Store the code on GitHub

Lab 2: Part 2 · Create a repository

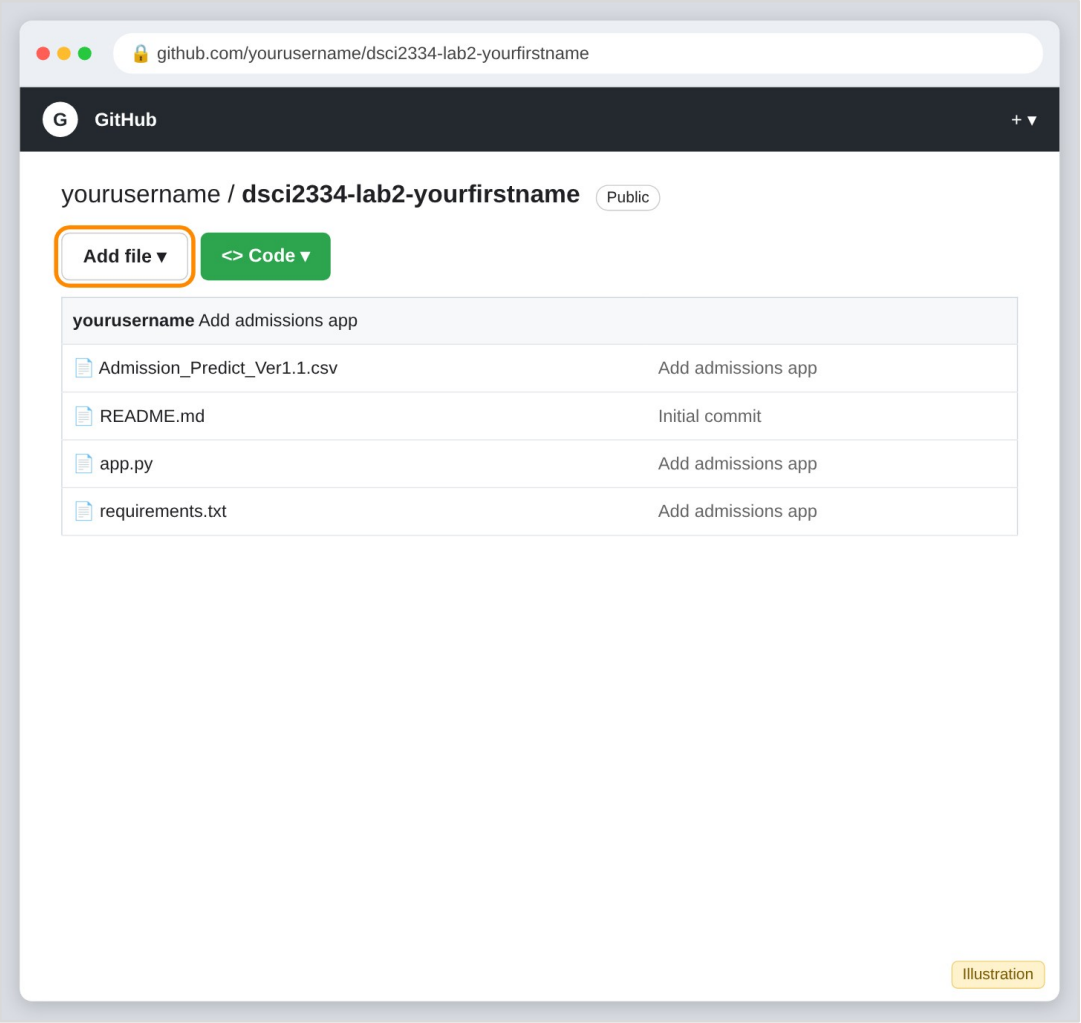
1. Sign in at github.com and click + › New repository.
2. Name it dsci2334-lab2-yourfirstname and choose Public.
3. Tick Add a README file and click Create repository.



Note: GitHub is a SaaS: you use the finished app without managing servers.

Lab 2: Part 2 · Upload the three files

1. In your repository, click Add file › Upload files.
2. Drag in app.py, requirements.txt and the CSV.
3. Commit message: “Add admissions app”. Click Commit changes.



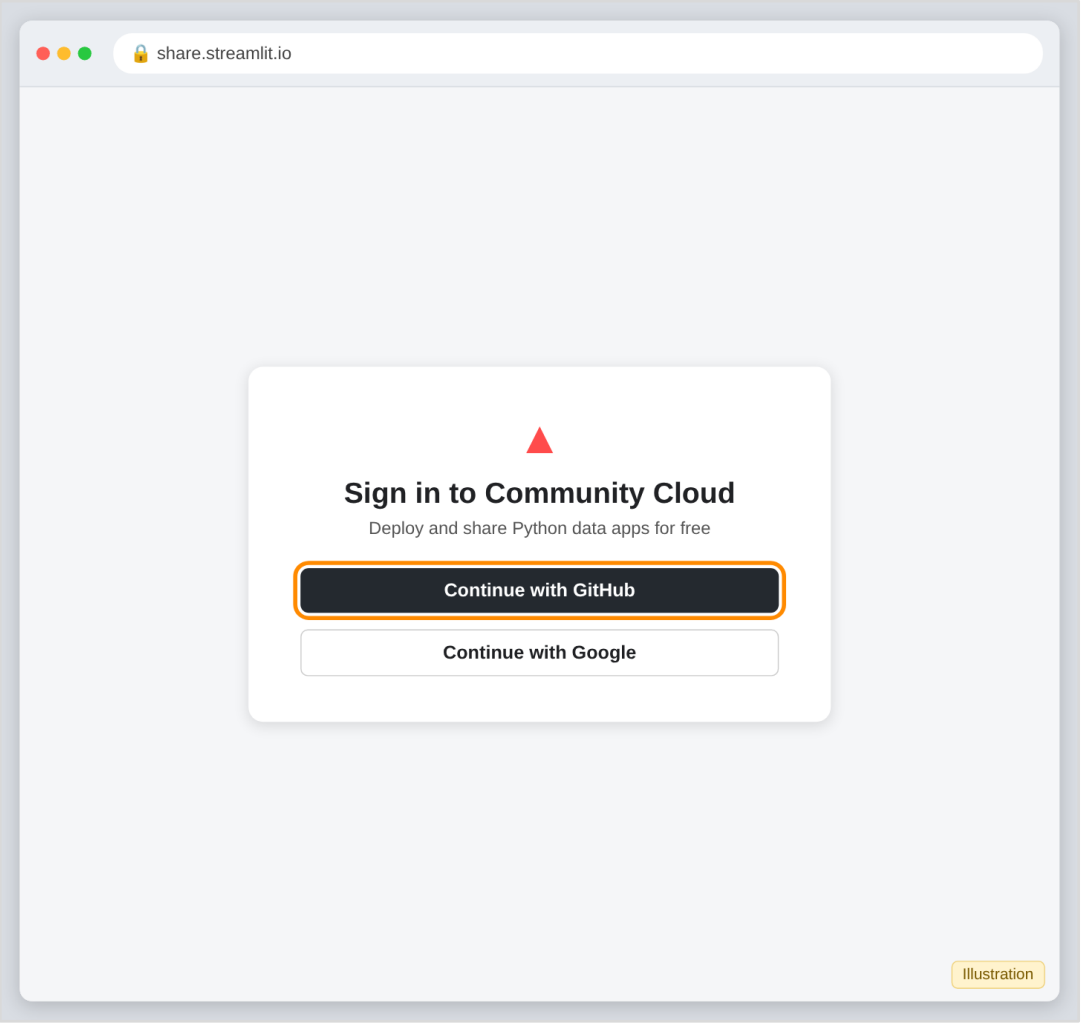
Note: Anyone can read a public repository. Never upload passwords or private data.



Lab 2: Part 3 · Deploy on a PaaS

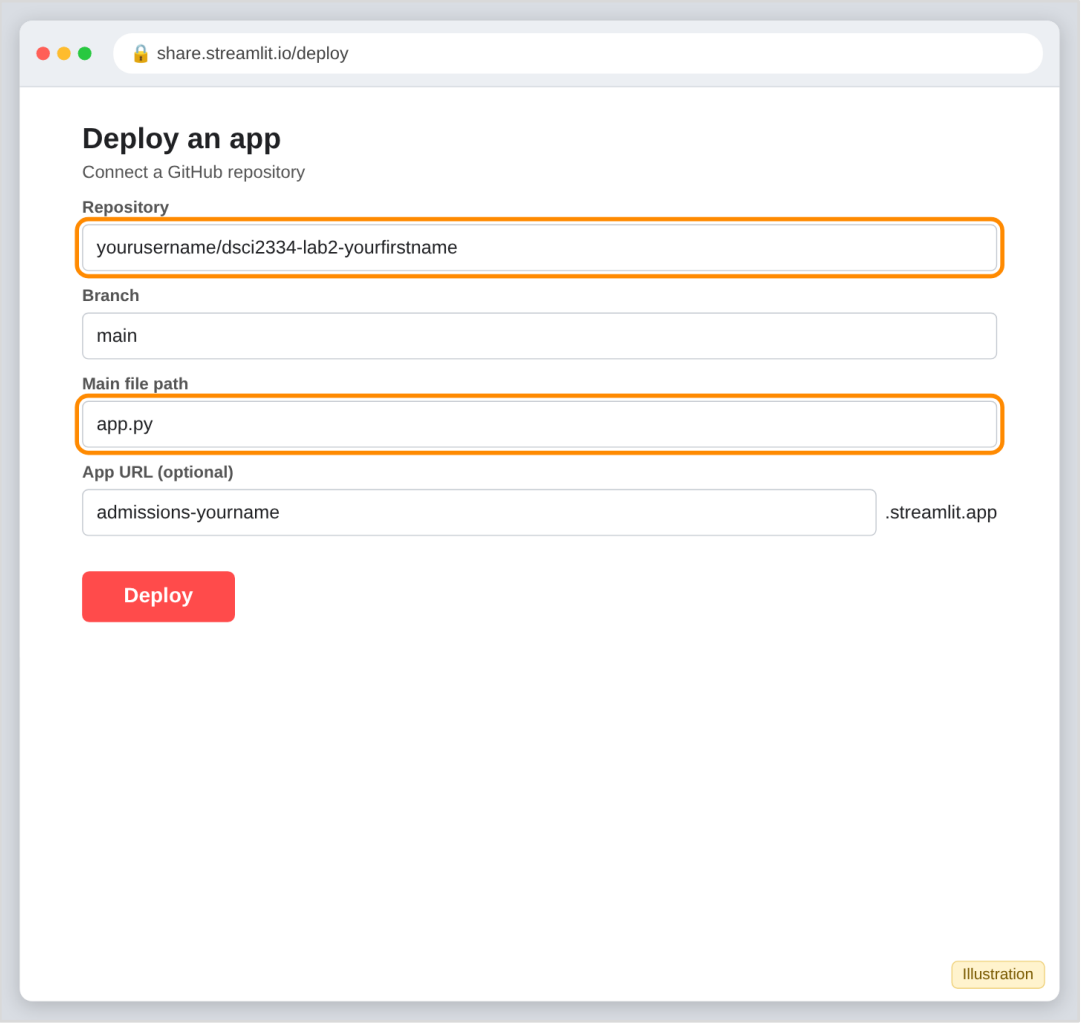
Lab 2: Part 3 · Sign in to Streamlit Community Cloud

1. Go to share.streamlit.io.
2. Click Continue with GitHub and sign in with the same account.
3. Allow Streamlit to read your repositories.



Lab 2: Part 3 · Deploy the app

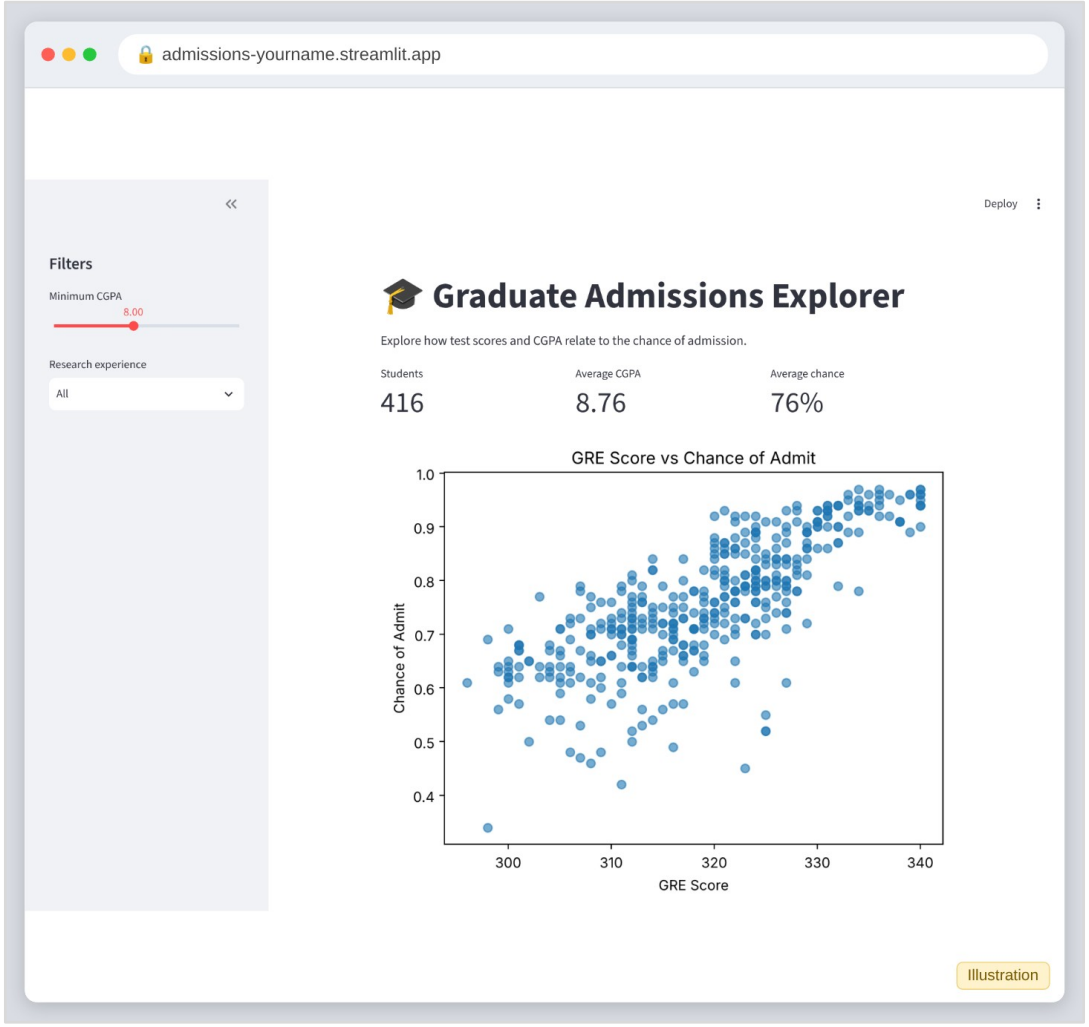
1. Click Create app and choose to deploy from a GitHub repository.
2. Repository: your repository. Branch: main. Main file path: app.py.
3. Optional: choose an App URL such as admissions-yourname. Click Deploy.



Note: You gave only code and settings — the platform picks the server, OS and Python.

Lab 2: Part 3 · Watch it build and open

1. Wait 1–3 minutes while the platform installs the packages from requirements.txt.
2. Your app opens at an address ending in .streamlit.app.
3. Check the numbers: Students 416, Average CGPA 8.76, Average chance 76%.
4. Open the link on your phone too.





Lab 2: Part 4 · Update and redeploy

Lab 2: Part 4 · Update and redeploy

Exercise 1 — Add a histogram

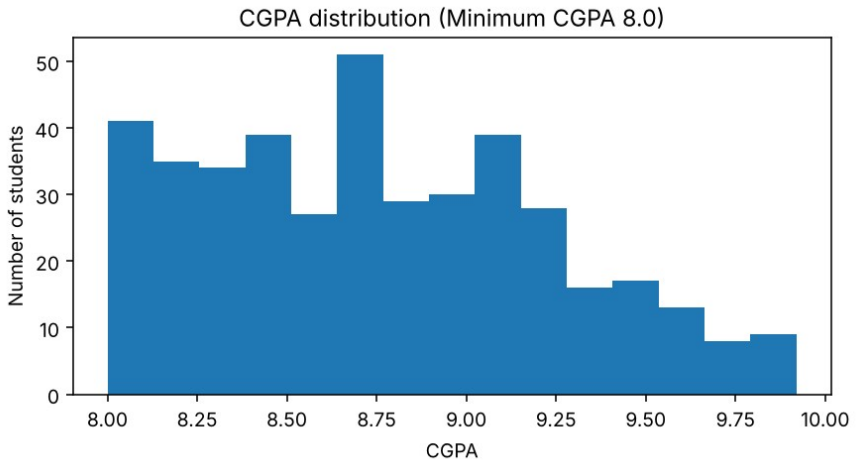
On GitHub, edit app.py (pencil icon) and add a histogram of CGPA just after st.pyplot(fig). Commit and wait about a minute.

Solution

```
st.subheader("CGPA distribution")
fig2, ax2 = plt.subplots()
ax2.hist(view["CGPA"], bins=15)
ax2.set_xlabel("CGPA")
ax2.set_ylabel("Number of students")
st.pyplot(fig2)
```

Lab 2: Part 4 · Update and redeploy — output

Output



Lab 2: Part 4 · Update and redeploy

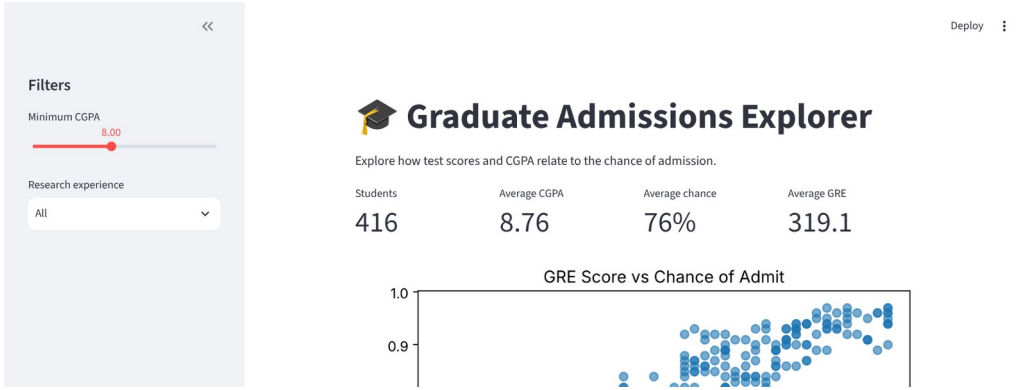
Exercise 2 — Add a fourth metric

Change the metrics row to four columns and add “Average GRE”. Commit and check your app.

Solution

```
c1, c2, c3, c4 = st.columns(4)
c1.metric("Students", len(view))
if len(view) > 0:
    c2.metric("Average CGPA", f"{view['CGPA'].mean():.2f}")
    c3.metric("Average chance", f"{view['Chance of Admit'].mean():.0%}")
    c4.metric("Average GRE", f"{view['GRE Score'].mean():.1f}")
```

Output



Lab 2: Part 4 · Update and redeploy

Exercise 3 — Change the default filter

Make the Minimum CGPA slider start at 9.0 instead of 8.0. What do the four metrics show now?

Solution

```
min_cgpa = st.sidebar.slider("Minimum CGPA", 6.5, 10.0, 9.0, 0.1)
```

Output

```
Students 144   ·   Average CGPA 9.31   ·   Average chance 88%   ·   Average GRE 328.2
```



Lab 2: Part 5 · Who managed what?

Lab 2: Part 5 · Who managed what?

Exercise 4 — Responsibility table (PaaS column of the stack in Lesson 1.4)

Copy the table and write You or Platform for each layer. The solution is shown in the right-hand column.

Layer	Solution	Why
Application code (app.py)	You	You uploaded and edited it
Data (CSV file)	You	You chose and uploaded it
Python packages	You list · Platform installs	requirements.txt tells the platform what to install
Python runtime and operating system	Platform	You never chose or patched them
Servers, storage and network	Platform	No server was created by you
Public web address (URL)	Platform	The .streamlit.app link was given to you

Lab 2: Bonus (optional) · Run the app on your own computer

Only if Python is installed. Put the three files in one folder, open a terminal there and run:

Code

```
python -m venv .venv
.venv\Scripts\activate          # macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
streamlit run app.py
```

Output

```
Local URL: http://localhost:8501      ← stop the app with Ctrl + C

Now YOU manage Python and the packages — compare that with the PaaS.
```



Lab 2: Part 6 · Reflection 💬

Lab 2: Part 6 · Reflection 💬

1. Which service model is Streamlit Community Cloud, and which is GitHub? Give one reason each.
2. List two jobs you did NOT have to do because you used a PaaS instead of your own server.
3. Why must requirements.txt be in the repository for the deployment to work?

Summary

- You deployed code without creating or managing any server — that is PaaS.
- Changing the code on GitHub redeployed the app automatically.
- Your code and data are still your responsibility, even on a platform.

Next: Lab 3 — estimate the cost of a cloud workload with the AWS Pricing Calculator (Lessons 1.7–1.8).

Lab 2: Reflection — model answers

Answer

1

1. Service models

Streamlit Community Cloud is PaaS: you bring the code, it provides the runtime, servers and URL. GitHub is SaaS: a finished application you use in the browser.

Answer

2

2. Jobs you did not do

Any two: buy or rent a server, install the OS, install Python, configure a web server, open firewall ports, get a domain/URL, patch software.

Answer

3

3. Why requirements.txt?

The platform builds a fresh environment for your app. requirements.txt tells it which packages (streamlit, pandas, matplotlib) to install; without it the app fails with ModuleNotFoundError.

Lab checklist

- [ ] Created the repository and uploaded the three files
- [ ] Deployed the app and opened it on a phone
- [ ] Completed Exercises 1–3 (histogram, fourth metric, default 9.0)
- [ ] Completed the responsibility table (Exercise 4)
- [ ] Answered the 3 reflection questions
- [ ] Submitted the app URL and repository link

NITA
الأكاديمية الوطنية لتقنية المعلومات
NATIONAL IT ACADEMY

Thank You